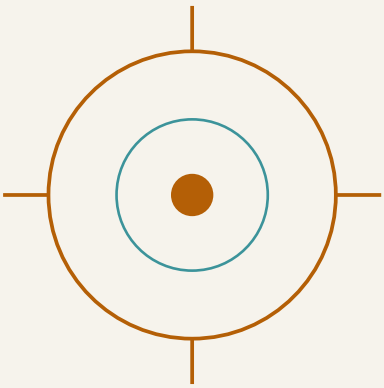
Esper

Archivio personale con ricerca semantica

Guida all'architettura e agli strumenti

Concetti, flussi di lavoro e componenti software



Documento di riferimento del progetto | stato: fase di progettazione

Indice

1 Visione d'insieme **2**

2 Concetti fondamentali **2**

3 Il flusso di inserimento (ingestion) **5**

4 Il flusso di ricerca **5**

5 Gli strumenti, uno per uno **6**

6 Schema del database **9**

7 Serve addestrare modelli? **10**

8 Interazione con un LLM (RAG) **10**

9 Il guscio desktop: Tauri **10**

10 Roadmap **11**

11 Glossario **11**

12 Per approfondire **12**

1 Visione d'insieme

Esper è un archivio personale di documenti con **ricerca semantica**: si descrive a parole ciò che si cerca e l'app trova i file pertinenti, anche se non contengono quelle parole esatte. Questo documento spiega i concetti e ogni strumento usato, così da capire che cosa succede sotto il cofano.

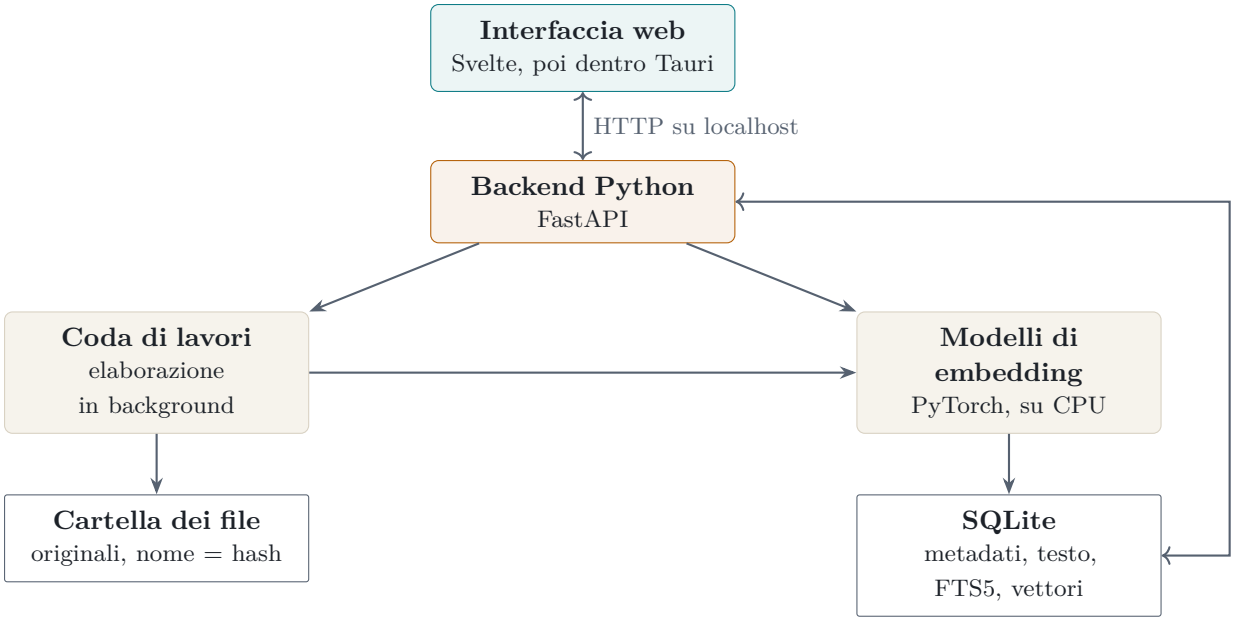


Figura 1: Architettura a livelli di Esper.

Tre idee reggono tutto:

1. **I file originali stanno su disco**, non nel database. Il database contiene solo informazioni *sui* file.
2. **Ogni file viene trasformato in testo** (o in un'immagine da analizzare), spezzato in pezzi, e **ogni pezzo diventa un vettore** (embedding).
3. **La ricerca confronta vettori** e li combina con la classica ricerca per parole chiave e con le etichette.

2 Concetti fondamentali

2.1 Che cos'è un embedding

Un embedding è una **lista di numeri** (un vettore, per esempio di 1024 elementi) che rappresenta il *significato* di un testo. Viene prodotto da una rete neurale addestrata con un obiettivo preciso: **testi con significato simile devono ricevere vettori vicini**, testi diversi vettori lontani.

Esempio concreto (le dimensioni reali sono molte di più):

Testo	Vettore	Rispetto al primo
il gatto dorme sul divano	`[0.21, -0.40, 0.77, ...]`	—
un felino riposa sul sofà	`[0.19, -0.38, 0.74, ...]`	vicino
fattura n. 4521 del 3 marzo	`[-0.62, 0.15, -0.20, ...]`	lontano

Le prime due frasi non condividono quasi nessuna parola, ma i loro vettori sono vicini. Una ricerca per parole chiave non le collegherebbe mai; la ricerca semantica sì.

Attenzione

> Non bisogna immaginare che ogni numero abbia un significato leggibile ("la terza dimensione è la felinità"). Il significato sta nella **posizione relativa** dei vettori nello spazio.

2.2 Come si misura la "vicinanza"

Si usa la **similarità del coseno**, cioè l'angolo tra due vettori **a** e **b**:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

- coseno ≈ 1: stessa direzione, significato molto simile;
- coseno ≈ 0: ortogonali, non correlati;
- coseno < 0: opposti (raro nei modelli di testo).

Se i vettori sono normalizzati (lunghezza 1), il coseno coincide con il **prodotto scalare**, molto veloce da calcolare. La maggior parte dei modelli moderni produce già vettori normalizzati.

2.3 Come nasce un modello di embedding

1. Si parte da un **Transformer** (la stessa famiglia degli LLM, in versione "encoder"): legge tutto il testo in una volta e, grazie al meccanismo di *attention*, ogni parola viene rappresentata tenendo conto di quelle attorno.
2. Il testo viene prima spezzato in **token** (pezzi di parola) da un *tokenizer*.
3. L'encoder produce un vettore per ogni token; un passaggio di *pooling* (media, oppure il token speciale `[CLS]`) li riduce a **un solo vettore per l'intero testo**.
4. L'addestramento è **contrastivo**: si danno al modello coppie di testi che vanno insieme (domanda e risposta pertinente, titolo e articolo, frase e traduzione) e coppie che non c'entrano. La funzione di perdita spinge i primi vettori ad avvicinarsi e i secondi ad allontanarsi.

Tu **non** farai questa parte: userai modelli già addestrati, scaricati una volta e usati in locale.

2.4 Perché non basta un embedding per documento

Un modello ha una **finestra massima** di token e, soprattutto, comprime tutto in un vettore di dimensione fissa. Un documento di 50 pagine ridotto a un solo vettore diventa una "media sfocata" di tutti gli argomenti, e una ricerca specifica non lo troverebbe.

Per questo si fa **chunking**: il documento viene diviso in pezzi (circa 300–500 token, con un po' di sovrapposizione fra pezzi adiacenti) e **ogni chunk ha il suo embedding**. In ricerca si trovano i chunk migliori e poi si risale al documento a cui appartengono.

2.5 Ricerca semantica e ricerca per parole chiave

	Semantica (vettori)	Parole chiave (BM25)
Punti di forza	Sinonimi, parafrasi, domande in linguaggio naturale, lingue diverse	Nomi propri, codici, numeri, termini rari, citazioni esatte
Debolezze	Può mancare "FT-2024-0381" o un cognome raro	Non capisce che "auto" e "macchina" sono vicini

Nessuna delle due basta da sola. La **ricerca ibrida** le usa entrambe e ne fonde i risultati.

2.6 BM25 e Reciprocal Rank Fusion

BM25

È l'algoritmo classico di ranking per parole chiave. Premia i documenti che contengono i termini cercati, soprattutto se rari (idea di TF-IDF), compensando la lunghezza del documento. SQLite lo offre già dentro FTS5.

Reciprocal Rank Fusion (RRF)

Fonde più classifiche in una. Ogni risultato riceve, per ogni lista, il punteggio $\frac{1}{k + \text{posizione}}$ con k di solito pari a 60, e i punteggi si sommano. Un risultato che compare in alto in *entrambe* le liste vince. Il vantaggio è che non serve confrontare punteggi di natura diversa (coseno e BM25), ma solo le posizioni.

2.7 Ricerca dei vicini: esatta e approssimata

- **Esatta (brute force)**: si confronta la domanda con tutti i vettori. Semplice e precisa; per decine o centinaia di migliaia di chunk su un solo computer è ancora accettabile.
- **Approssimata (ANN)**: indici come HNSW o IVF evitano di guardare tutto, con un piccolo margine di errore. Servono quando i vettori sono milioni.

Per Esper all'inizio il brute force è sufficiente.

2.8 Che cos'è davvero il RAG

RAG significa **Retrieval-Augmented Generation**, in due fasi:

1. **Retrieval**: trova i passaggi rilevanti (è la ricerca semantica di Esper).
2. **Generation**: passa quei passaggi a un LLM insieme alla domanda, e il modello risponde basandosi su di essi, citando le fonti.

In breve

> Esper nasce come *solo retrieval*. La parte "G" si aggiunge dopo, senza modificare il resto.

3 Il flusso di inserimento (ingestion)

Quando trascini un file nell'app:



1. **Salvataggio**: si calcola l'hash del contenuto (SHA-256). Se esiste già, il file è un duplicato e non si reinserisce. Altrimenti viene copiato nella cartella dell'app con nome basato sull'hash e si crea una riga nel database con stato `in_coda`.
2. **Estrazione** (dipende dal formato):
   - `txt`, `md`: si legge direttamente.
   - `pdf`: PyMuPDF estrae il testo pagina per pagina. Se una pagina non contiene testo (scansione), la si trasforma in immagine e si passa all'OCR.
   - `jpg`, `png`: OCR per il testo contenuto, più embedding visivo dell'immagine stessa.
3. **Chunking**: il testo si divide in pezzi rispettando per quanto possibile i confini naturali (titoli, paragrafi). Ogni chunk conserva la pagina o la posizione di origine.
4. **Embedding**: i chunk passano in blocchi (*batch*) al modello, che restituisce un vettore per chunk. È la fase più pesante sulla CPU.
5. **Indicizzazione**: testo, vettori e metadati vengono scritti nel database. Lo stato diventa `indicizzato` (o `errore`, con il motivo).

La pipeline gira **in background**: il salvataggio del file è immediato, l'elaborazione procede in una coda e la UI mostra lo stato.

4 Il flusso di ricerca

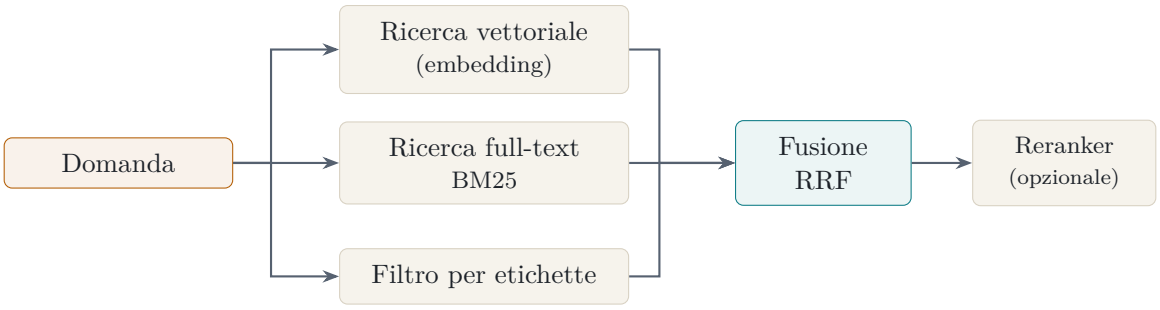


Figura 2: Dalla domanda ai risultati.

1. La domanda viene trasformata in embedding **con lo stesso modello** usato per i documenti. È fondamentale: vettori di modelli diversi non sono confrontabili.
2. Si cercano i chunk più vicini (vettoriale) e quelli con le parole giuste (BM25).

3. I filtri per etichetta restringono l'insieme dei candidati.
4. RRF fonde le liste.
5. Opzionale: un **reranker** (modello più lento e preciso) rilegge i migliori 20–50 candidati insieme alla domanda e li riordina.
6. I chunk vengono raggruppati per documento e mostrati con un frammento di anteprima.

Prefissi dei modelli

> Alcuni modelli (la famiglia E5, per esempio) vogliono un prefisso diverso per la domanda e per il documento (`query:` e `passage:`). Va usato il modello con le sue convenzioni, altrimenti la qualità cala.

5 Gli strumenti, uno per uno

5.1 Python 3.12 e uv

- **Python 3.12**: scelto perché le librerie di machine learning (PyTorch, ONNX Runtime, ...) hanno pacchetti già pronti per questa versione. La 3.14 del sistema è troppo recente.
- **uv**: gestore di ambienti e pacchetti. Crea il `.venv`, installa da PyPI, scarica versioni di Python e blocca le versioni esatte in `uv.lock`, così l'ambiente è riproducibile.
- `uv run <comando>`: esegue un comando *dentro* l'ambiente del progetto senza doverlo attivare e, se serve, prima sincronizza le dipendenze.

5.2 FastAPI (backend)

Framework web per Python che espone il backend come **API HTTP** (per esempio `POST /documents`, `GET /search?q=...`).

- Usa le *type hint* di Python per validare i dati in ingresso e generare la documentazione interattiva (`/docs`).
- Gira sopra **Uvicorn**, il server in ascolto su `localhost`.
- La UI è un normale client HTTP: questo separa nettamente interfaccia e logica, ed è ciò che permette di aggiungere Tauri dopo senza riscrivere nulla.

5.3 SQLite

Database relazionale contenuto **in un solo file**, senza server da avviare. È in ogni telefono e browser: affidabile e molto testato. Per Esper contiene:

- la tabella dei documenti e delle etichette;
- il testo dei chunk;
- l'indice full-text (FTS5);
- i vettori (con sqlite-vec).

Il backup consiste nel copiare un file (insieme alla cartella dei file originali). Va attivata la

modalità **WAL** (`PRAGMA journal_mode=WAL`), che permette letture mentre un'altra connessione scrive: utile perché la pipeline scrive in background mentre la UI legge.

5.4 FTS5 (ricerca full-text)

Modulo di SQLite per la ricerca per parole chiave. Costruisce un **indice invertito** (per ogni parola, l'elenco dei chunk che la contengono) e offre la funzione di ranking **BM25**. Si può configurare un *tokenizer* che ignora maiuscole e accenti (`unicode61 remove_diacritics 2`); per l'italiano è comodo anche lo *stemming* (ridurre le parole alla radice).

5.5 sqlite-vec (vettori in SQLite)

Estensione di SQLite che aggiunge un tipo di tabella virtuale per memorizzare vettori e fare ricerche di vicinanza (distanza coseno o L2). Fa **ricerca esatta (brute force)**, adeguata per un archivio personale. È un progetto giovane (pre-1.0): l'API può cambiare, quindi va fissata la versione.

Alternativa: LanceDB, database vettoriale embedded con indici ANN, più adatto se i vettori crescono molto. Si può passare a esso più avanti: i vettori si possono ricalcolare o esportare.

5.6 PyMuPDF (PDF)

Libreria Python basata su MuPDF. Estrae testo (con posizione e struttura), immagini e metadati dai PDF molto velocemente. Può anche **renderizzare una pagina come immagine**, utile per le pagine scansionate da mandare all'OCR.

5.7 Tesseract (OCR)

Motore OCR open source: trasforma immagini di testo in testo. Richiede il pacchetto di sistema `tesseract` e i dati della lingua (`tesseract-langpack-ita` su Fedora). Funziona bene su testo stampato pulito; su manoscritti o foto storte i risultati sono scadenti. Si usa da Python con `pytesseract` o `ocrmypdf`.

5.8 Alternative all'OCR

Esistono OCR basati su reti neurali (EasyOCR, PaddleOCR, Docling) più robusti ma più pesanti. Per iniziare, Tesseract basta.

5.9 PyTorch e sentence-transformers

- **PyTorch**: la libreria di calcolo tensoriale su cui girano i modelli. Si installa la versione **CPU-only** (senza CUDA), molto più leggera.
- **sentence-transformers**: incapsula il caricamento dei modelli di embedding (da Hugging Face), il tokenizer, il pooling e la normalizzazione. Con poche righe, `model.encode(lista_di_testi)` restituisce i vettori.

- **Hugging Face Hub**: sito da cui i modelli si scaricano la prima volta e poi restano in cache locale (`~/.cache/huggingface`). Dopo il download Esper funziona **offline**.

Sulla CPU conviene elaborare in **batch**, usare tutti i core (PyTorch lo fa da solo) e limitare la lunghezza massima dei chunk. In alternativa, **ONNX Runtime** può essere più veloce con modelli esportati e quantizzati (int8), a scapito di una configurazione più complessa.

5.10 Modelli di embedding di testo

Modello	Vettore	Dimensione	Note
bge-m3 (BAAI)	1024	circa 570M parametri (∼2 GB)	Multilingue, contesto fino a 8192 token; produce anche rappresentazioni sparse e multi-vettore. Ottimo, ma il più pesante.
multilingual-e5-base / small	768 / 384	280M / 120M	Più leggero, buono in italiano; richiede i prefissi `query:`/`passage:`.
nomic-embed-text	768	∼137M	Leggero; il supporto per l'italiano è meno forte.

Consiglio: partire con **bge-m3** e passare a e5-small se la velocità su CPU è insufficiente. Il codice va scritto in modo che il modello sia **sostituibile da configurazione**.

Cambiare modello

> Cambiare modello significa **ricalcolare tutti gli embedding**. Per questo nel database si salva il nome del modello usato per ogni vettore. I valori della tabella vanno verificati sulle rispettive pagine Hugging Face prima di fissarli nel codice.

5.11 Modelli per le immagini (CLIP / SigLIP)

Modelli *multimodali* con due encoder (uno per le immagini, uno per i testi) addestrati insieme perché una foto e la sua descrizione finiscano vicine nello **stesso spazio**. Permettono ricerche come "tramonto sul mare" su fotografie senza alcun testo. **SigLIP** (Google) è una variante di CLIP con una funzione di perdita migliore; esistono versioni multilingue.

Due spazi vettoriali

> Lo spazio di CLIP/SigLIP è **diverso** da quello di bge-m3. Servono due indici separati: in ricerca si interrogano entrambi (la domanda viene codificata con ciascuno dei due modelli) e i risultati si fondono con RRF.

5.12 Reranker (opzionale)

Un **cross-encoder** legge insieme domanda e candidato e produce un punteggio di pertinenza. È molto più accurato degli embedding (che codificano domanda e documento separatamente), ma molto più lento, e per questo si applica solo ai migliori candidati. Esempio: `bge-reranker-v2-m3`. Si aggiunge solo se la qualità della ricerca ibrida non basta.

5.13 Interfaccia: Svelte

Framework JavaScript per costruire la UI come componenti, qui con **Vite** (strumento di sviluppo e build) e TypeScript. È già scritta in versione demo, con dati simulati, in `frontend/`. Nulla nel backend dipende da questa scelta: la UI parla con FastAPI secondo il contratto descritto in `docs/API_CONTRACT.md`.

5.14 Docling / Unstructured (opzionale)

Librerie che trasformano documenti complessi (PDF con tabelle, colonne, layout) in testo strutturato. Utili se i PDF hanno impaginazione difficile; per iniziare, PyMuPDF basta.

6 Schema del database

Bozza, che potrà cambiare:

```
-- Un record per ogni file importato
documents(
  id            INTEGER PRIMARY KEY,
  sha256        TEXT UNIQUE,   -- identifica il contenuto, evita i duplicati
  original_name TEXT,
  mime_type     TEXT,
  size_bytes    INTEGER,
  stored_path   TEXT,          -- dove sta il file nella cartella dell'app
  status        TEXT,          -- in_coda | in_elaborazione | indicizzato | errore
  error         TEXT,
  created_at    TEXT
)

tags(id INTEGER PRIMARY KEY, name TEXT UNIQUE)
document_tags(document_id, tag_id)   -- relazione molti-a-molti

-- Un record per ogni pezzo di testo
chunks(
  id          INTEGER PRIMARY KEY,
  document_id INTEGER,
  position    INTEGER,         -- ordine nel documento
  page        INTEGER,         -- per i PDF
  text        TEXT
)

chunks_fts   -- tabella virtuale FTS5 sul testo dei chunk (parole chiave)
chunks_vec   -- tabella virtuale sqlite-vec: un vettore per chunk
images_vec   -- vettori visivi (SigLIP), una riga per immagine

embedding_models(id, name, dimensions)  -- quale modello ha prodotto i vettori
```

7 Serve addestrare modelli?

No, non per iniziare, e probabilmente mai. Il progetto usa modelli *pre-addestrati* a cui si danno solo input. Si distinguono tre livelli:

1. **Uso diretto** (è ciò che faremo): scarico il modello, lo eseguo, ottengo vettori. Nessun addestramento.
2. **Fine-tuning**: riaddestrare leggermente un modello sui propri dati (per esempio coppie "domanda → documento corretto" del proprio archivio) per migliorarlo su un dominio specifico. Si considera solo se, dopo aver misurato, la ricerca fallisce in modo sistematico, e richiede un insieme di esempi etichettati.
3. **Addestramento da zero**: fuori scala (richiede enormi quantità di dati e calcolo). Non ha senso qui.

Cosa conviene fare davvero: valutare

> Costruire un piccolo insieme di prova (30–50 coppie "domanda → documento giusto") e misurare quanto spesso il documento giusto compare tra i primi risultati (**recall@k**). Con questo si confrontano modelli e parametri di chunking in modo oggettivo, non a sensazione.

8 Interazione con un LLM (RAG)

Fase successiva. Il flusso:

1. La ricerca restituisce i migliori chunk per la domanda.
2. Si costruisce un prompt: istruzioni, chunk numerati e domanda.
3. L'LLM genera una risposta che cita i chunk usati.

Due modi di far girare l'LLM:

API (OpenRouter)
: Un unico servizio con molti modelli e un'API compatibile con quella di OpenAI; si paga a consumo e conviene scegliere modelli economici. I chunk lasciano il computer: questa è la scelta che sacrifica la privacy.

Locale (Ollama + Gemma)
: **Ollama** scarica ed esegue modelli in locale con un'API semplice. Gemma è una famiglia di modelli compatti di Google. Tutto resta sul computer, ma su CPU un modello piccolo è lento e meno capace.

Il backend può avere un'interfaccia comune, così da cambiare provider con un parametro di configurazione. Il comportamento giusto è far rispondere *solo* in base ai passaggi forniti e dire "non lo trovo" se non ci sono, per limitare le invenzioni (allucinazioni).

9 Il guscio desktop: Tauri

Tauri è un framework per impacchettare una UI web come applicazione desktop.

- La UI gira nella **webview del sistema operativo** (WebKitGTK su Linux), non in un Chromium incluso come in Electron. Il risultato è un'app molto più piccola e leggera.
- Il nucleo nativo è in **Rust**; per questo progetto è poco più di configurazione.
- Il backend Python viene lanciato come **sidecar** (programma accessorio avviato e chiuso insieme all'app).
- Produce installer nativi (AppImage, .deb, .rpm).
- Difficoltà principale: **impacchettare Python con le librerie ML** (PyInstaller o simili) è pesante e delicato. Per questo si rimanda: si sviluppa prima il backend con la UI nel browser, e Tauri arriva a fine percorso.

10 Roadmap

Fase	Contenuto
1	Backend FastAPI, SQLite, importazione di txt/md/pdf, chunking, embedding con bge-m3, ricerca ibrida, etichette
2	OCR per PDF scansionati e immagini; embedding visivo con SigLIP; ricerca unificata
3	Interfaccia web curata (anteprime, filtri, stato dell'elaborazione) — *prototipo già realizzato con dati simulati*
4	Pacchetto desktop con Tauri
5	LLM: risposte con citazioni (OpenRouter e/o Ollama + Gemma)
6	Audio (Whisper per la trascrizione), eventuale cloud e sincronizzazione

11 Glossario

g Vettore numerico che rappresenta il significato di un testo o di un'immagine.

Pezzo di testo (parola o parte di parola) su cui lavora un modello.

er Architettura di rete neurale basata sull'attention; base di LLM ed embedding moderni.

Parte del transformer che *legge* un testo e lo rappresenta (usata per gli embedding).

Riduzione dei vettori dei token a un unico vettore per il testo intero.

Pezzo di documento su cui si calcola un embedding.

del coseno Misura di vicinanza tra vettori in base all'angolo.

e / ANN Ricerca esatta contro approssimata dei vettori più vicini (HNSW, IVF).

Algoritmo di ranking per parole chiave.

Modulo di ricerca full-text di SQLite.

Tecnica per fondere più classifiche di risultati.

Modello (cross-encoder) che riordina i candidati leggendo domanda e testo insieme.

Ricerca di passaggi più generazione di una risposta con un LLM.

Modello linguistico di grandi dimensioni (genera testo).

g Ulteriore addestramento di un modello già addestrato.

Riconoscimento del testo in un'immagine.

Programma esterno avviato insieme all'app (in Tauri).

zione Ridurre la precisione dei pesi di un modello (per esempio int8) per renderlo più veloce e leggero.

Percentuale di volte in cui il risultato giusto è tra i primi k.

12 Per approfondire

Ordine di lettura suggerito, dal più utile al meno urgente:

1. Documentazione di **sentence-transformers** (sbert.net): capire `encode`, modelli e prefissi.
2. Pagina del modello **BAAI/bge-m3** su Hugging Face (model card).
3. Documentazione di **SQLite FTS5** e del repository **sqlite-vec**.
4. Tutorial ufficiale di **FastAPI**.
5. Articolo originale *Attention Is All You Need* (Transformer), per chi vuole la teoria.
6. Articoli di SigLIP e CLIP per la ricerca di immagini.
7. Documentazione di **Tauri v2** (sezione sidecar), quando arriverà la fase 4.